Bitácora de Práctica - LAB-09: Encapsulamiento

- **Integrantes:**
  - Velasco Gomez Iker Alejandro
  - Medina Reyes Miguel Ángel

1. Objetivos de la Práctica

- Aplicar de forma rigurosa el principio de encapsulamiento en el lenguaje Java mediante el uso de modificadores de visibilidad privados (`private`), garantizando el aislamiento y la protección del estado interno de los objetos.
- Desarrollar e integrar lógica de validación e invariantes en constructores, métodos de acceso (`getters`) y de mutación (`setters`) para interceptar y rechazar transiciones de estado lógicamente inválidas o no autorizadas.

2. Arquitectura y Clases Desarrolladas

El modelo de dominio se estructuró bajo el paquete modular `mazmorra`, aplicando separación de responsabilidades y ocultamiento de atributos:

Componente	Tipo	Responsabilidad y Mecanismo de Encapsulamiento
`Herramienta`	Clase Concreta	Encapsula atributos de `nombre`, `daño` (acotado contractualmente a un valor máximo de 50) y `durabilidad`. Implementa mecanismos seguros de lectura y reducción gradual de vida útil tras su uso.

Componente	Tipo	Responsabilidad y Mecanismo de Encapsulamiento
`Inventario`	Clase Concreta	Administra una colección interna acotada a una capacidad máxima de 3 herramientas. Protege el acceso indexado verificando límites para impedir desbordamientos y exposiciones de punteros nulos.
`Personaje`	Clase Abstracta	Centraliza la gestión de `vidaActual`, `vidaMaxima`, `inventario` y referencia al `compañero`. Define operaciones protegidas contra mutaciones directas mediante rutinas de `recibirDanio()` y `curar()`.
`Caballero`, `Mago`, `Arquero`	Subclases	Especializan la clase base configurando umbrales de vida máxima y perfiles de resistencia específicos según el rol funcional de cada entidad dentro del sistema.

3. Pruebas y Validación de Seguridad

Para auditar la robustez del diseño, se ejecutaron suites de prueba desde el paquete `trampas`, divididas en dos fases operativas:

3.1. Pruebas de Acceso Léxico y Modificadores

A través de `TrampasDeAcceso.java`, se verificó la inaccesibilidad directa de atributos marcados con visibilidad `private` desde contextos foráneos. Todas las tentativas de

asignación directa dispararon de forma deliberada y satisfactoria errores de compilación (`The field is not visible`), confirmando el aislamiento perimetral de los datos.

3.2. Pruebas de Reglas de Negocio e Integridad

Mediante la clase de prueba `Tramposo.java`, se forzaron condiciones de frontera y datos anómalos. El sistema mitigó con éxito los siguientes escenarios de vulneración:

- **Inyección de daño negativo:** Intentos de causar daño con valores negativos para revertir el cálculo y curar al objetivo fueron interceptados, descartando la operación o forzando la normalización del argumento.
- **Sobrecarga de salud:** Intentos de curación por encima del límite estricto de `vidaMaxima` fueron neutralizados, manteniendo el valor fijado en el techo permitido.
- **Armamento fuera de escala:** La instanciación de herramientas con atributos de daño superiores a 50 fue bloqueada desde el constructor, asignando valores acotados por defecto.
- **Desbordamiento de inventario:** El intento de inserción de un cuarto ítem en un inventario con capacidad para 3 elementos fue rechazado sin comprometer la estructura de almacenamiento existente.
- **Indexación inválida:** Solicitudes de consulta o equipamiento en índices fuera de rango o vacíos arrojaron advertencias de control interno, impidiendo fallos catastróficos de ejecución (`IndexOutOfBoundsException`).
- **Ejecución de ataque sin arma:** Se evaluaron y contuvieron las condiciones donde un personaje intentaba ejecutar un cálculo de daño ofensivo sin disponer de una herramienta asignada en el inventario.
- **Resurrección no autorizada:** El sistema bloqueó la aplicación de efectos curativos sobre instancias cuya vida había alcanzado el valor cero (estado inactivo/muerto).

```
trampas\TrampasDeAcceso.java:10: error: vida has private access in Personaje
        heroe.vida = 9999;
              ^
trampas\TrampasDeAcceso.java:11: error: inventario has private access in Personaje
        heroe.inventario = new Inventario();
              ^
trampas\TrampasDeAcceso.java:14: error: durabilidad has private access in Herramienta
        espada.durabilidad = 1000000;
               ^
trampas\TrampasDeAcceso.java:17: error: herramientas has private access in Inventario
        bolsa.herramientas[0] = espada;
              ^
trampas\TrampasDeAcceso.java:18: error: cuantas has private access in Inventario
        bolsa.cuantas = 50;
              ^
```

4. Conclusiones

La implementación estricta del principio de encapsulamiento demostró ser un mecanismo crítico para la robustez del software:

- **Inviolabilidad del estado interno:** El uso de modificadores `private` suprimió vectores de manipulación directa de los atributos por parte de entidades externas (`TrampasDeAcceso`), forzando cualquier interacción a través de la interfaz pública controlada.
- **Preservación de invariantes:** Las rutinas de saneamiento y comprobación de límites integradas en métodos mutadores y constructores blindaron la lógica de negocio frente a flujos malformados (`Tramposo`), garantizando consistencia determinista tanto en ejecuciones convencionales (`Main`) como bajo condiciones adversas.